EXPERIMENT 10 - MINI PROJECT

Dataset→Random Forest→K-Means→Flask→Web App

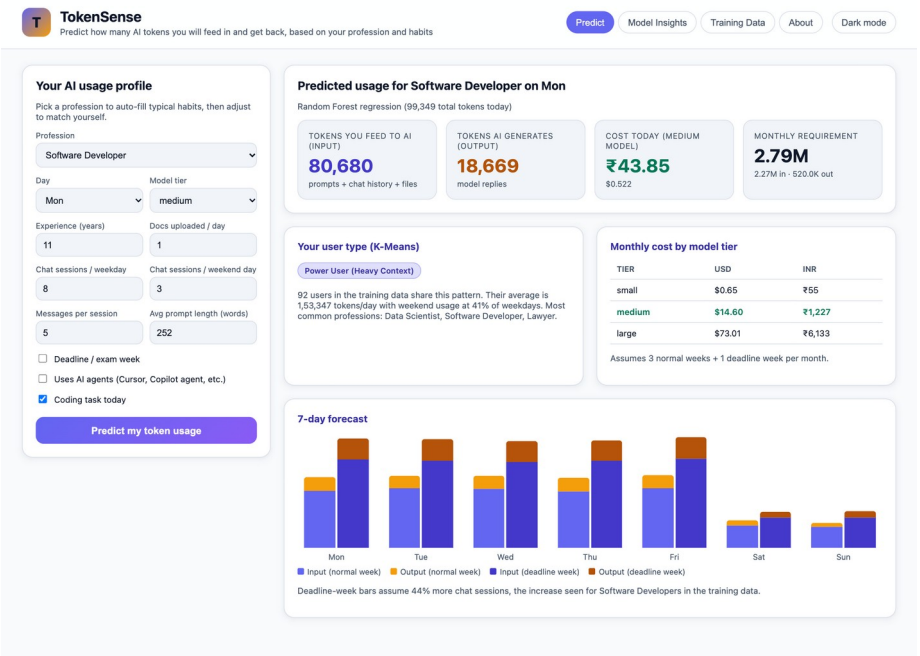
TokenSense – AI Token Utilization Predictor

- ➢ Your Full Name
- ➢ TE IT B - Roll No.
- ➢ Machine Learning Lab

TokenSense is a machine learning web application that predicts how many **AI tokens** a person will use per day and per month, based on their **profession** and AI usage habits. It predicts two values: **input tokens**, which the user feeds to the AI (prompts, chat history and uploaded files), and **output tokens**, which the AI generates as replies. It also estimates the **cost** on small, medium and large AI models and tells the user what **type of AI user** they are.

A **Random Forest Regression** model (Module 3) predicts the daily input and output tokens, and **K-Means Clustering** (Module 4) groups users into four usage types. A **Flask** backend serves the trained models through a REST API, and an **HTML, CSS and JavaScript** frontend lets the user enter their profile and see the prediction, cost, user type and a 7-day forecast.

GitHub Repo URL - https://github.com/jerry5379/tokensense



Aim -

To build a web application that predicts the AI token requirement of a person according to their profession and usage habits, split into input tokens and output tokens, estimates the cost, and groups users into usage types using machine learning.

Problem Statement -

AI assistants such as ChatGPT, Claude, Gemini and GitHub Copilot are now used every day by developers, students, writers, teachers, doctors and lawyers, and all of them bill usage in **tokens** (about 0.75 words per token). People cannot easily tell how many tokens they will need in a month or which model tier they can afford. Usage depends on the **profession** (a developer pastes code and logs, a content writer asks for long replies), on **habits** (chats per day, messages per chat, prompt length, uploaded files, AI agents) and on the **calendar** (weekends and deadline or exam weeks). TokenSense learns these patterns from data and gives each person a personal token budget.

Training Dataset -

Real per-user token logs are private, so a realistic **synthetic dataset** was generated in Python (src/generate_data.py, random seed 42) using the rules of LLM billing: every new message re-sends the whole conversation, so input tokens grow roughly with the square of the messages per session; each uploaded document adds about 2,500 tokens; AI agents multiply the context about 2.5 times; and ±12% random noise is added.

- **File:** data/token_usage_daily.csv, **9,000 rows** (300 users × 30 days), 12 professions.
- **Features (11):** profession, day of week, model tier, experience, deadline/exam week, uses AI agents, chat sessions per day, messages per session, prompt length (words), documents uploaded, coding task.
- **Targets (2):** input_tokens and output_tokens.
- **Split:** 7,200 rows for training, 1,800 rows for testing (80/20).

Sample of the training data (15 of 9,000 rows):

User	Profession	Day	Deadline	Agents	Sessions	Msgs/ session	Prompt words	Docs	Input tokens	Output tokens
U001	Software Developer	Sat	0	0	4	6	268	0	60,996	10,805
U021	Researcher	Thu	1	0	11	7	175	5	1,08,947	17,824
U041	Digital Marketer	Tue	0	0	12	7	157	2	1,85,534	32,819
U061	Software Developer	Wed	0	0	2	8	328	3	63,236	8,830
U081	Researcher	Wed	1	0	8	8	288	3	1,94,399	21,365
U101	Digital Marketer	Mon	0	0	5	3	111	3	15,835	6,923
U121	Software Developer	Wed	0	0	4	8	234	1	89,685	11,518

User	Profession	Day	Deadline	Agents	Sessions	Msgs/ session	Prompt words	Docs	Input tokens	Output tokens
U141	Researcher	Fri	1	0	15	5	232	8	1,16,425	38,005
U161	Digital Marketer	Wed	0	0	3	8	127	1	42,936	6,701
U181	Software Developer	Fri	0	0	8	8	232	2	1,76,851	22,596
U201	Researcher	Sat	1	0	1	6	383	1	23,874	2,059
U221	Digital Marketer	Fri	0	0	6	1	142	0	3,396	5,645
U241	Software Developer	Sat	0	0	5	6	312	1	80,505	20,039
U261	Researcher	Sun	0	0	4	6	308	3	81,760	8,880
U281	Digital Marketer	Sun	0	0	4	6	116	1	37,616	7,912

Algorithms Used -

1. Random Forest Regression (Module 3)

A Random Forest is an ensemble of many decision trees. Each tree is trained on a bootstrap sample of the rows and considers a random subset of features at each split, choosing the split that most reduces the squared error. The forest predicts the **average of all trees**, which reduces the overfitting of a single tree while still capturing non-linear patterns. This suits token usage, which is strongly non-linear: doubling the messages in a chat roughly quadruples the input tokens, and AI agents multiply everything.

One multi-output forest with **200 trees** predicts both targets. Categorical features are one-hot encoded with a ColumnTransformer inside a scikit-learn Pipeline, so the same preprocessing is used in training and in the web app. On unseen test data it achieved **$R^2 = 0.915$**:

Target	MAE (tokens)	RMSE (tokens)	R^2
input_tokens	11,298	26,145	0.924
output_tokens	2,116	3,557	0.906
overall	6,707	14,851	0.915

2. K-Means Clustering (Module 4)

K-Means is an unsupervised algorithm that divides data into k groups. It picks k starting centroids, assigns every point to the nearest centroid (Euclidean distance), moves each centroid to the mean of its points, and repeats until nothing changes. Each user is described by four standardised features from their 30 days: log of average daily tokens, burstiness (std / mean), output share, and weekend-to-weekday ratio. **k = 4** was chosen with the elbow method and silhouette score (0.25):

User type	Users	Avg tokens/day	Most common professions
Light / Irregular User	53	36,912	Digital Marketer, Designer, Business Analyst
Steady 9-to-5 Professional	98	44,607	Customer Support, Doctor, Teacher
Anytime Learner / Creator	57	88,156	Student, Researcher, Content Writer

User type	Users	Avg tokens/day	Most common professions
Power User (Heavy Context)	92	1,53,347	Data Scientist, Software Developer, Lawyer

Working together: for a new user, the Random Forest predicts all 7 days of the week, the same four features are calculated from that forecast, and K-Means assigns the user type.

Frontend -

A single-page web interface built with plain **HTML, CSS and JavaScript** (templates/index.html, static/style.css, static/app.js). The **Predict** tab has the input form and shows input tokens, output tokens, today's cost, monthly requirement, the K-Means user type, monthly cost per model tier and a 7-day forecast chart. The **Model Insights**, **Training Data** and **About** tabs show model metrics, clusters, training charts, dataset sample and architecture. Choosing a profession auto-fills its typical habits, and a Light/Dark mode button is provided.

Backend -

A Python **Flask** server (app.py). On start-up it loads the trained Random Forest pipeline and K-Means model (saved with joblib by src/train.py) and exposes a REST API that returns JSON: **POST /api/predict** (prediction, weekly and monthly forecast, cost, user type), **GET /api/insights** (metrics and clusters), **GET /api/meta** (form options and defaults) and **GET /api/dataset** (data preview).

Frontend–Backend Connection -

When the user clicks **Predict my token usage**, app.js reads the form and sends it as JSON to **/api/predict** using fetch(). Flask converts the JSON into a one-row pandas DataFrame, runs it through the same pipeline used in training (one-hot encoding, then Random Forest), runs K-Means on the weekly forecast, calculates the cost, and returns one JSON response. JavaScript renders it into cards, tables and charts without reloading the page. Training runs separately and only writes the model files the server reads, which keeps the ML code and the web code separate.

```
Browser (HTML + CSS + JavaScript)
     |  fetch() -> JSON over HTTP
     v
Flask REST API (app.py)  -->  models/rf_token_model.joblib  (Random Forest)
                              models/kmeans_users.joblib    (K-Means + scaler)
                              models/insights.json          (metrics, clusters)
```

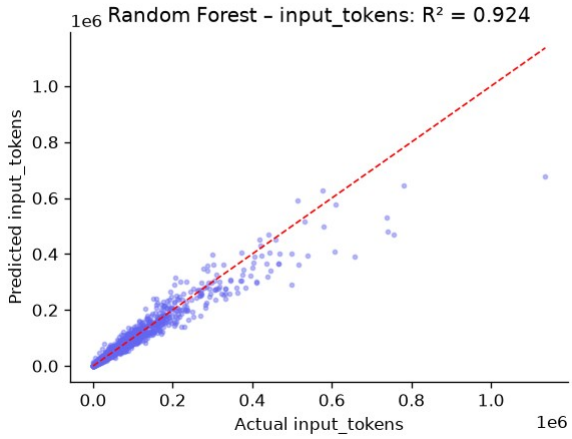
Generating Data & Training the Model -

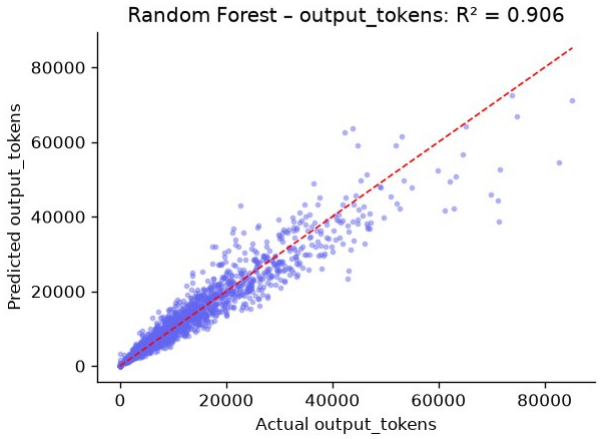
The dataset is generated and both models are trained from the terminal:

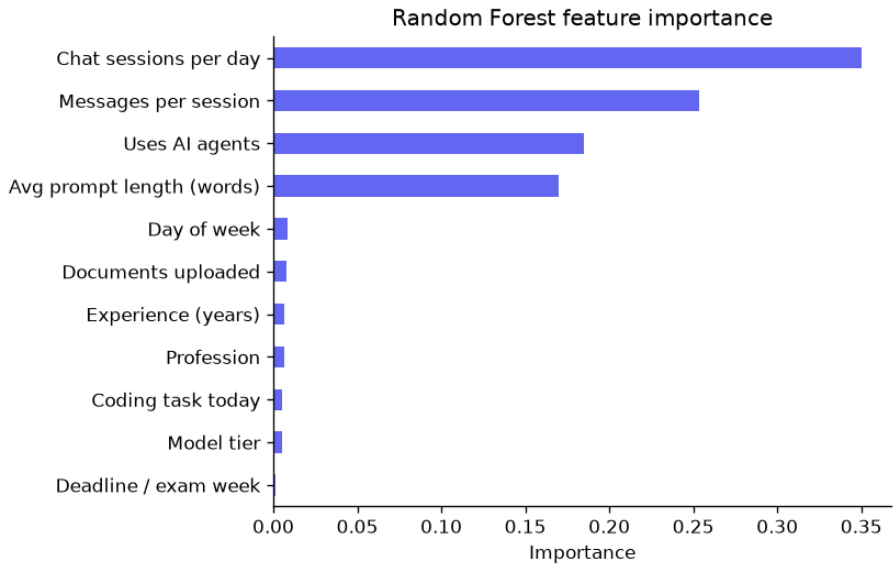
```
$ python src/generate_data.py
Wrote 9000 rows -> data/token_usage_daily.csv
                  input_tokens  output_tokens
profession
Doctor                 15850.0         3322.0
Designer               17742.0         4137.0
Teacher                40978.0         9010.0
Student                50630.0         8769.0
Lawyer                 54849.0         8119.0
Digital Marketer       62141.0        14160.0
Business Analyst       65972.0        10729.0
Customer Support       66615.0        14334.0
Researcher            106604.0        13914.0
Content Writer        107555.0        26087.0
Data Scientist        137447.0        17210.0
Software Developer    140669.0        21590.0

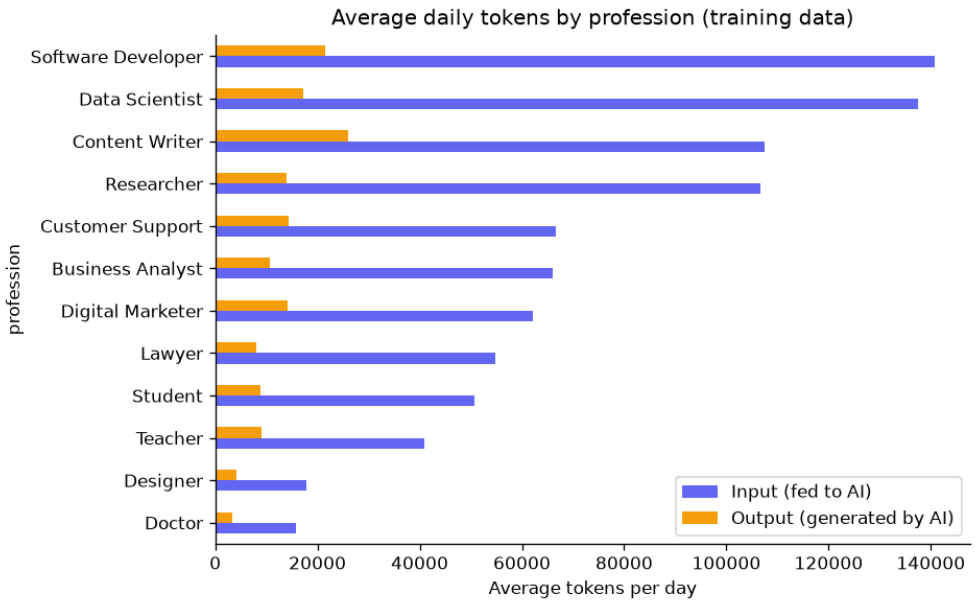
$ python src/train.py
Loaded 9000 rows
[RF] input_tokens   MAE=  11297.8  RMSE=  26144.7  R2=0.9239
[RF] output_tokens  MAE=   2116.4  RMSE=   3556.6  R2=0.906
[RF] overall        MAE=   6707.1  RMSE=  14850.7  R2=0.9149
[K-Means] k=4, silhouette=0.2547
          Steady 9-to-5 Professional     users= 98 avg/day=  44607
          Power User (Heavy Context)     users= 92 avg/day= 153347
          Anytime Learner / Creator      users= 57 avg/day=  88156
          Light / Irregular User         users= 53 avg/day=  36912
Saved models to models and figures to static/figures
```

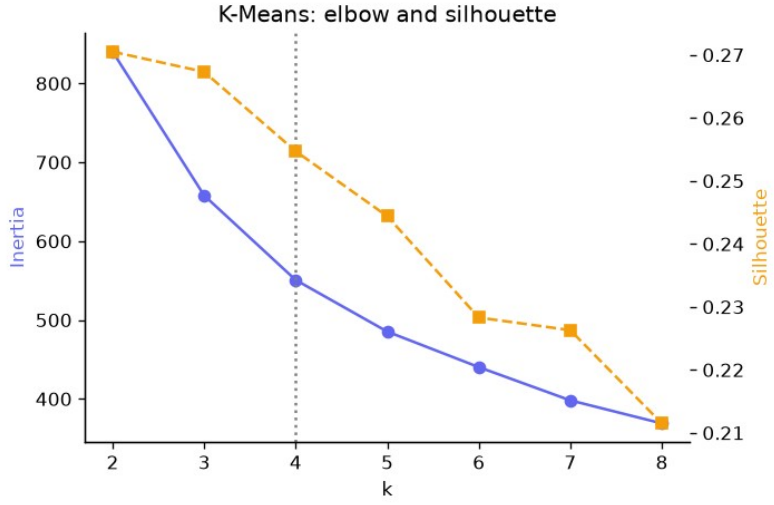
Model Results -

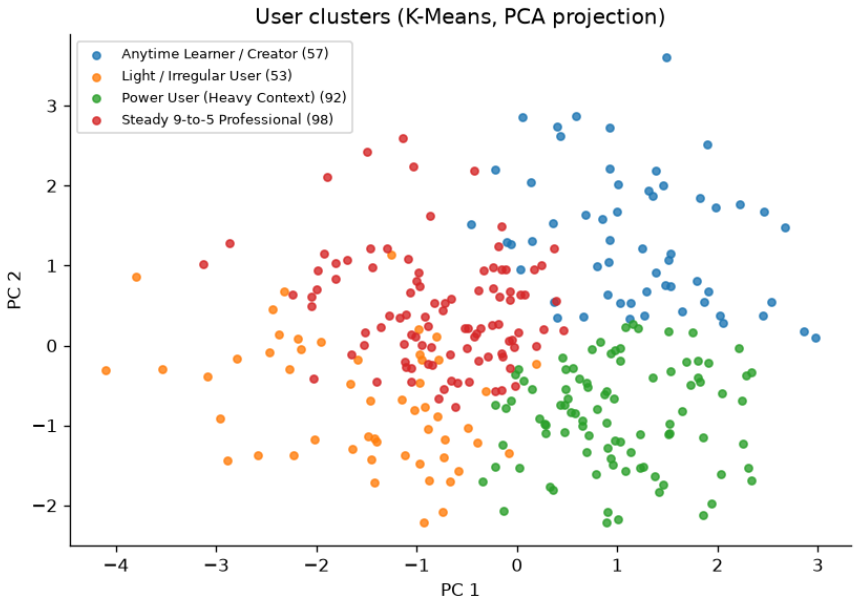





Random Forest feature importance
Chat sessions per day
Messages per session
Uses AI agents
Avg prompt length (words)
Day of week
Documents uploaded
Experience (years)
Profession
Coding task today
Model tier
Deadline / exam week
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
Importance


Average daily tokens by profession (training data)
profession
Software Developer
Data Scientist
Content Writer
Researcher
Customer Support
Business Analyst
Digital Marketer
Lawyer
Student
Teacher
Designer
Doctor
Input (fed to AI)
Output (generated by AI)
0
20000
40000
60000
80000
100000
120000
140000
Average tokens per day

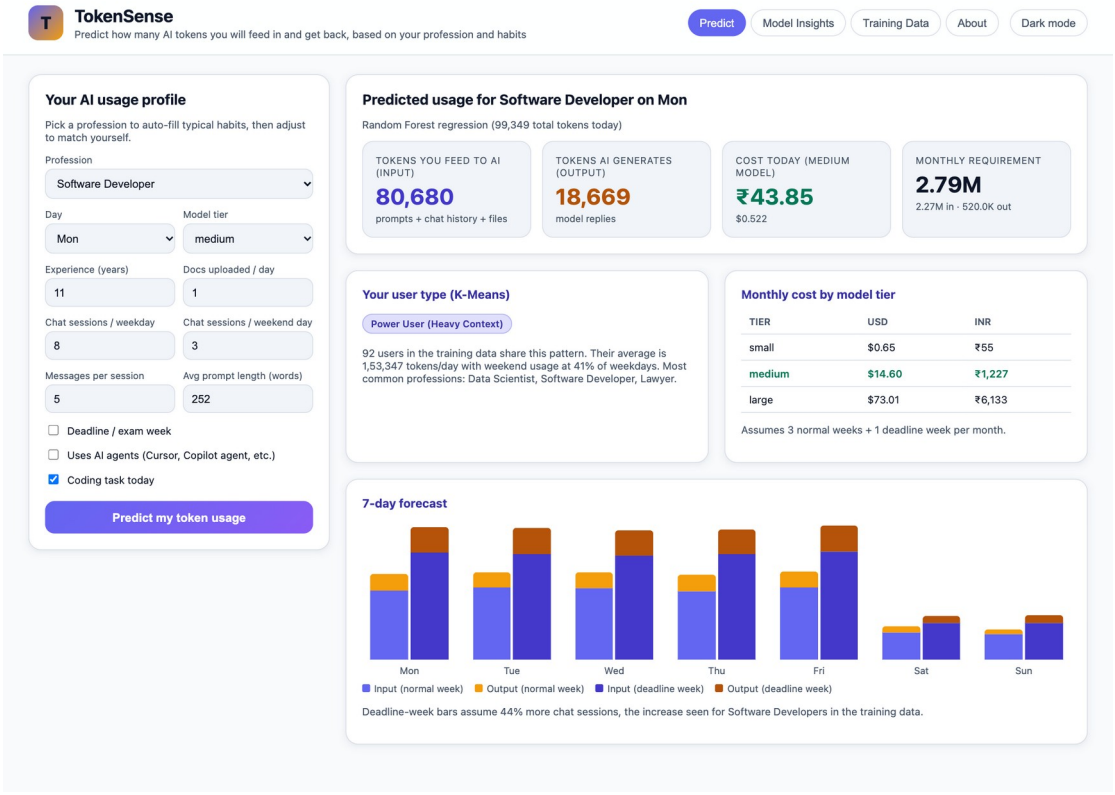

K-Means: elbow and silhouette
Inertia
800
700
600
500
400
Silhouette
0.27
0.26
0.25
0.24
0.23
0.22
0.21
2
3
4
5
6
7
8
k



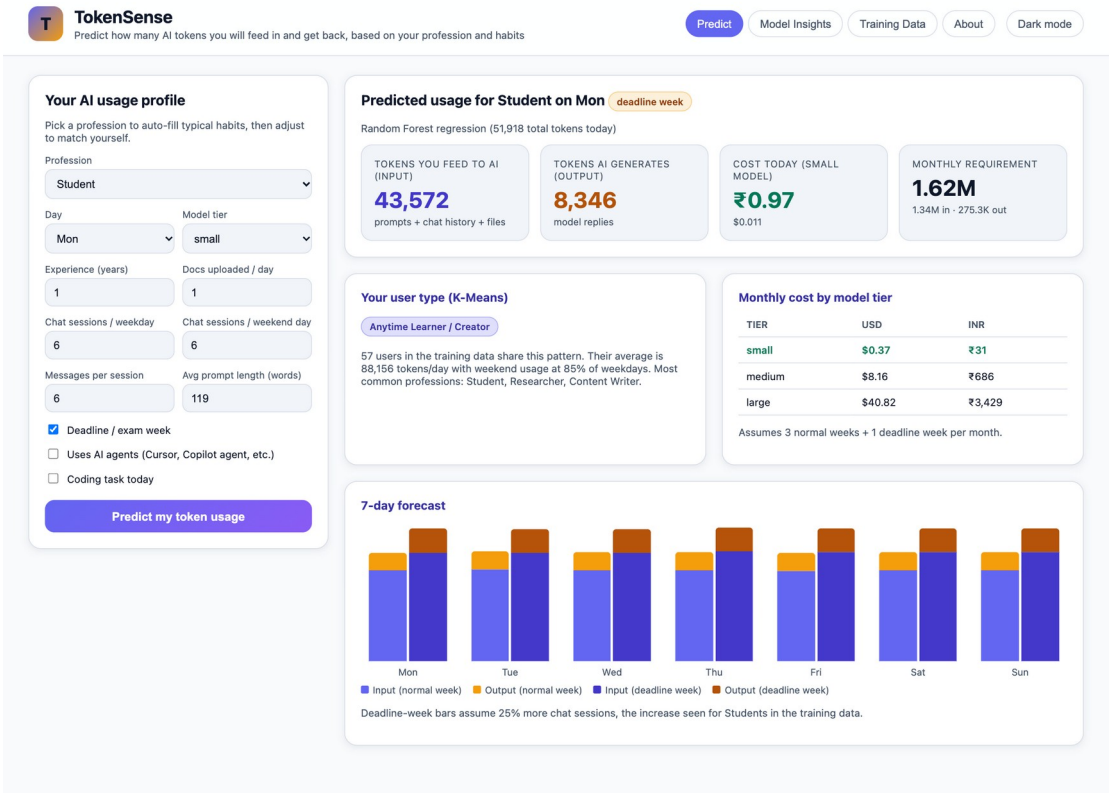
Running the Web App -

```
$ source .venv/bin/activate
$ python app.py
 * Serving Flask app 'app'
 * Running on http://127.0.0.1:5050
Press CTRL+C to quit
```

Predict – Software Developer:



Predict – Student in exam week:



Model Insights:

TokenSense
Predict how many AI tokens you will feed in and get back, based on your profession and habits

Predict | Model Insights | Training Data | About | Dark mode

Random Forest regression · test set results

TARGET	MAE	RMSE	R^2
input_tokens	11,298	26,145	0.924
output_tokens	2,116	3,557	0.906
overall	**6,707**	**14,851**	**0.915**

200 trees. Trained on 7,200 rows, tested on 1,800 unseen rows (80/20 split).

Random Forest · feature importance

Chat sessions per day	35.0%
Messages per session	25.3%
Uses AI agents	18.5%
Avg prompt length (words)	17.0%
Day of week	0.8%
Documents uploaded	0.8%
Experience (years)	0.7%
Profession	0.6%
Coding task today	0.5%
Model tier	0.5%
Deadline / exam week	0.2%

K-Means user types (k = 4, silhouette 0.2547)

Steady 9-to-5 Professional
98 users · 44,607 tokens/day · output share 23% · weekend 45% of weekday · burstiness 0.629
Top professions: Customer Support (17), Doctor (13), Teacher (13)

Power User (Heavy Context)
92 users · 1,53,347 tokens/day · output share 13% · weekend 41% of weekday · burstiness 0.625
Top professions: Data Scientist (18), Software Developer (16), Lawyer (12)

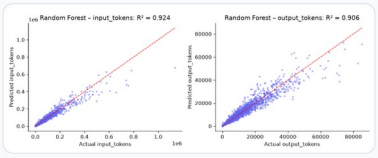
Anytime Learner / Creator
57 users · 88,156 tokens/day · output share 16% · weekend 85% of weekday · burstiness 0.541
Top professions: Student (23), Researcher (15), Content Writer (7)

Light / Irregular User
53 users · 36,912 tokens/day · output share 19% · weekend 39% of weekday · burstiness 0.83
Top professions: Digital Marketer (11), Designer (10), Business Analyst (10)

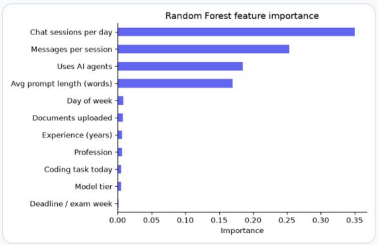
Training figures



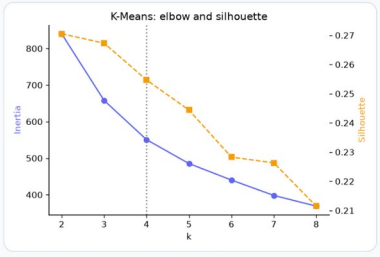
Average daily input/output tokens by profession (training data)



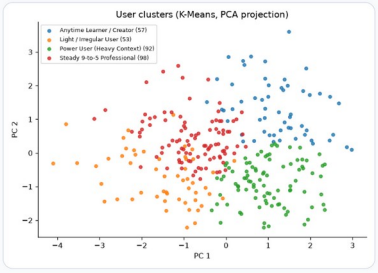
Random Forest: actual vs predicted on the 20% test set



Random Forest: feature importance

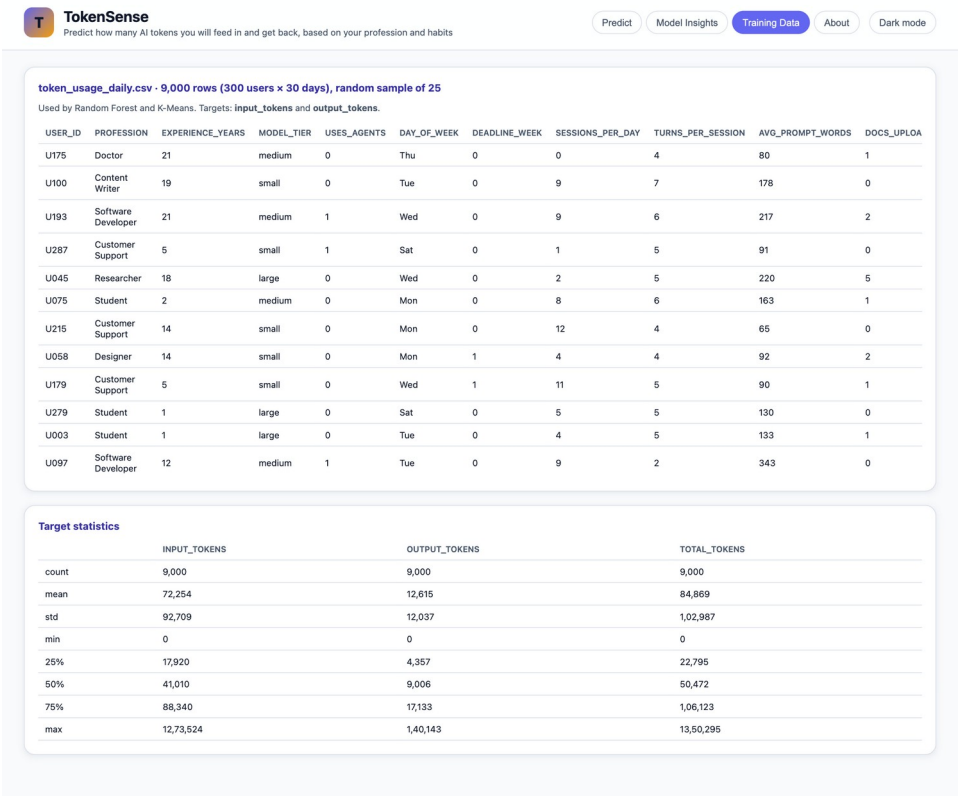


K-Means: choosing k with elbow and silhouette



K-Means: user clusters projected with PCA

Training Data:



TokenSense
Predict how many AI tokens you will feed in and get back, based on your profession and habits

Predict | Model Insights | Training Data | About | Dark mode

token_usage_daily.csv · 9,000 rows (300 users × 30 days), random sample of 25

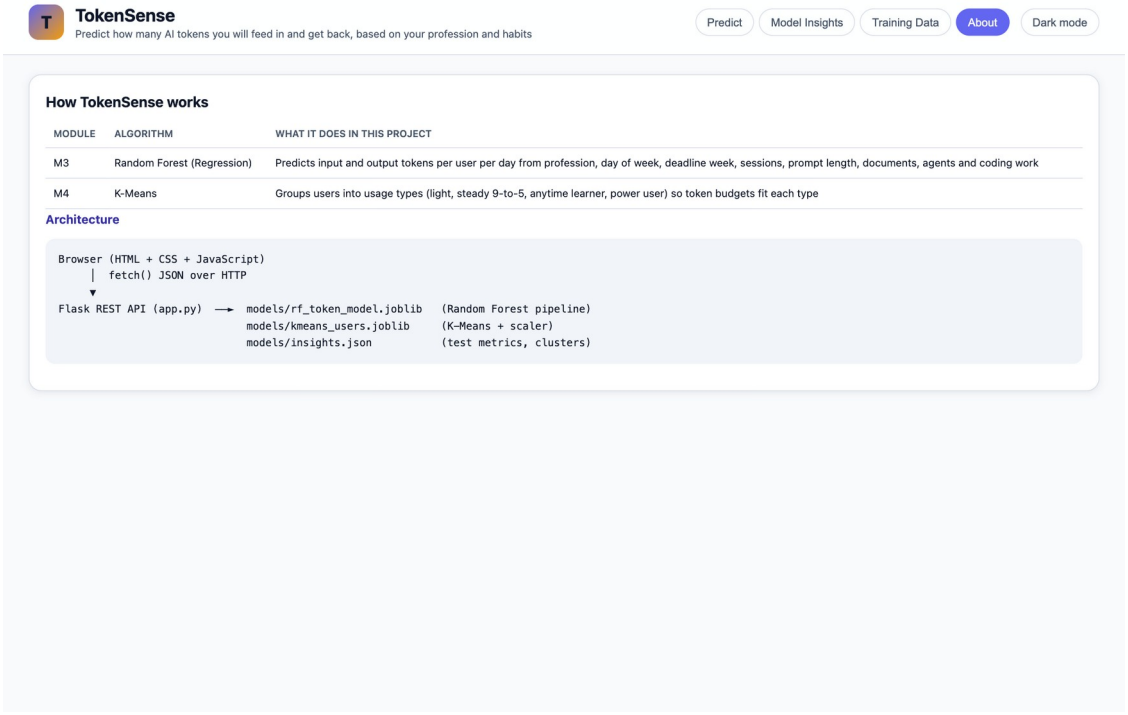
Used by Random Forest and K-Means. Targets: **input_tokens** and **output_tokens**.

USER_ID	PROFESSION	EXPERIENCE_YEARS	MODEL_TIER	USES_AGENTS	DAY_OF_WEEK	DEADLINE_WEEK	SESSIONS_PER_DAY	TURNS_PER_SESSION	AVG_PROMPT_WORDS	DOCS_UPLOA
U175	Doctor	21	medium	0	Thu	0	0	4	80	1
U100	Content Writer	19	small	0	Tue	0	9	7	178	0
U193	Software Developer	21	medium	1	Wed	0	9	6	217	2
U287	Customer Support	5	small	1	Sat	0	1	5	91	0
U045	Researcher	18	large	0	Wed	0	2	5	220	5
U075	Student	2	medium	0	Mon	0	8	6	163	1
U215	Customer Support	14	small	0	Mon	0	12	4	65	0
U058	Designer	14	small	0	Mon	1	4	4	92	2
U179	Customer Support	5	small	0	Wed	1	11	5	90	1
U279	Student	1	large	0	Sat	0	5	5	130	0
U003	Student	1	large	0	Tue	0	4	5	133	1
U097	Software Developer	12	medium	1	Tue	0	9	2	343	0

Target statistics

	INPUT_TOKENS	OUTPUT_TOKENS	TOTAL_TOKENS
count	9,000	9,000	9,000
mean	72,254	12,615	84,869
std	92,709	12,037	1,02,987
min	0	0	0
25%	17,920	4,357	22,795
50%	41,010	9,006	50,472
75%	88,340	17,133	1,06,123
max	12,73,524	1,40,143	13,50,295

About (algorithms and architecture):



TokenSense
Predict how many AI tokens you will feed in and get back, based on your profession and habits

Predict | Model Insights | Training Data | About | Dark mode

How TokenSense works

MODULE	ALGORITHM	WHAT IT DOES IN THIS PROJECT
M3	Random Forest (Regression)	Predicts input and output tokens per user per day from profession, day of week, deadline week, sessions, prompt length, documents, agents and coding work
M4	K-Means	Groups users into usage types (light, steady 9-to-5, anytime learner, power user) so token budgets fit each type

Architecture

```
Browser (HTML + CSS + JavaScript)
     |  fetch() JSON over HTTP
     ▼
Flask REST API (app.py)  ──►  models/rf_token_model.joblib   (Random Forest pipeline)
                              models/kmeans_users.joblib     (K-Means + scaler)
                              models/insights.json           (test metrics, clusters)
```

Important Code -

Random Forest training (src/train.py)

```
def train_random_forest(df):
    X, y = df[FEATURES], df[TARGETS]
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=SEED)


    pipe = Pipeline([
        ("prep", ColumnTransformer([
            ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), CATEGORICAL),
            ("num", "passthrough", NUMERIC),
        ])),
        ("model", RandomForestRegressor(n_estimators=N_ESTIMATORS, min_samples_leaf=2,
                                        random_state=SEED, n_jobs=-1)),
    ])
    pipe.fit(X_train, y_train)
    pred = pipe.predict(X_test)


    metrics = {
        t: {"mae": round(mean_absolute_error(y_test[t], pred[:, i]), 1),
            "rmse": round(root_mean_squared_error(y_test[t], pred[:, i]), 1),
            "r2": round(r2_score(y_test[t], pred[:, i]), 4)}
        for i, t in enumerate(TARGETS)
    }
    metrics["overall"] = {"mae": round(mean_absolute_error(y_test, pred), 1),
                          "rmse": round(root_mean_squared_error(y_test, pred), 1),
                          "r2": round(r2_score(y_test, pred), 4)}
    for k, m in metrics.items():
        print(f"[RF] {k:14s} MAE={m['mae']:>9}  RMSE={m['rmse']:>9}  R2={m['r2']}")


    # Feature importance, with one-hot columns summed back into their original feature
    names = pipe.named_steps["prep"].get_feature_names_out()
    importance = {}
    for name, imp in zip(names, pipe.named_steps["model"].feature_importances_):
        col = name.split("__", 1)[1]
        source = next((c for c in CATEGORICAL if col.startswith(c + "_")), col)
        importance[source] = importance.get(source, 0) + float(imp)
    importance = dict(sorted(((k, round(v, 4)) for k, v in importance.items()), key=lambda kv:
-kv[1]))


    return pipe, metrics, importance, (y_test, pred), (len(X_train), len(X_test))
```

K-Means clustering (src/train.py)

```
def train_kmeans(df):
    rows = []
    for uid, g in df.groupby("user_id"):
        prof = build_profile(g["day_of_week"], g["input_tokens"], g["output_tokens"])
        prof.update(user_id=uid, profession=g["profession"].iloc[0],
                    avg_daily_tokens=float((g["input_tokens"] + g["output_tokens"]).mean()))
        rows.append(prof)
    profiles = pd.DataFrame(rows)
    scaler = StandardScaler()
    Z = scaler.fit_transform(profiles[PROFILE_FEATURES])


    elbow = []
    for k in range(2, 9):
        km = KMeans(n_clusters=k, n_init=10, random_state=SEED).fit(Z)
        elbow.append({"k": k, "inertia": round(km.inertia_, 2),
                      "silhouette": round(silhouette_score(Z, km.labels_), 4)})


    km = KMeans(n_clusters=N_CLUSTERS, n_init=10, random_state=SEED).fit(Z)
    profiles["cluster"] = km.labels_
    centers = pd.DataFrame(scaler.inverse_transform(km.cluster_centers_), columns=PROFILE_FEATURES)
    names = name_clusters(centers)
    profiles["cluster_name"] = profiles["cluster"].map(names)
    silhouette = round(silhouette_score(Z, km.labels_), 4)
    ...
```

Backend prediction API (app.py)

```
@app.post("/api/predict")
def predict():
```

```
    body = request.get_json(force=True)
    try:
        row = parse_inputs(body)
    except (KeyError, ValueError) as e:
        return jsonify({"error": str(e)}), 400
    weekend_sessions = float(body.get("weekend_sessions", row["sessions_per_day"]))
    rf = state["rf"]


    # M3: Random Forest predicts today's input and output tokens
    x = pd.DataFrame([row])[FEATURES]
    inp, out = (max(0.0, v) for v in rf.predict(x)[0])


    normal = rf.predict(week_frame(row, weekend_sessions, 0)).clip(min=0)
    deadline = rf.predict(week_frame(row, weekend_sessions, 1)).clip(min=0)
    weekly = [{"day": d, "input": int(n[0]), "output": int(n[1]),
               "deadline_input": int(dl[0]), "deadline_output": int(dl[1])}
              for d, n, dl in zip(DAYS, normal, deadline)]


    # A typical month = 3 normal weeks + 1 deadline/exam week, scaled to 30 days
    month_in = (3 * normal[:, 0].sum() + deadline[:, 0].sum()) * 30 / 28
    month_out = (3 * normal[:, 1].sum() + deadline[:, 1].sum()) * 30 / 28
    monthly = {
        "input_tokens": int(month_in), "output_tokens": int(month_out),
        "total_tokens": int(month_in + month_out),
        "cost_by_tier": {t: cost(month_in, month_out, t) for t in MODEL_TIERS},
    }


    # M4: K-Means assigns the user type from their predicted week
    km = state["kmeans"]
    profile = build_profile(DAYS, normal[:, 0], normal[:, 1])
    z = km["scaler"].transform(pd.DataFrame([profile])[PROFILE_FEATURES])
    cluster_id = int(km["kmeans"].predict(z)[0])
    cluster = next(c for c in state["insights"]["kmeans"]["clusters"] if c["cluster"] == cluster_id)


    return jsonify({
        "inputs": row,
        "prediction": {"input_tokens": int(inp), "output_tokens": int(out), "total_tokens": int(inp
+ out),
                       "cost": cost(inp, out, row["model_tier"])},
        "weekly": weekly,
        "deadline_uplift": state["deadline_uplift"][row["profession"]],
        "monthly": monthly,
        "cluster": {**cluster, "profile": {k: round(v, 3) for k, v in profile.items()}},
    })
```

Frontend call to the backend (static/app.js)

```
async function onPredict(e) {
  e.preventDefault();
  const btn = $("#predict-form button");
  btn.disabled = true;
  try {
    renderResults(await api("/api/predict", { method: "POST", body: readForm() }));
  } catch (err) {
    toast(err.message, true);
  } finally {
    btn.disabled = false;
  }
}
```

Conclusion -

TokenSense predicts the daily input and output AI tokens of a person with $R^2 = 0.915$ using Random Forest Regression, and groups users into four meaningful usage types with K-Means, without any labels. Combined with a Flask backend and a simple web frontend, it gives a practical tool for planning AI usage and cost for every profession. Future scope: train on real usage logs, add time-series forecasting, and connect directly to AI providers' usage APIs.